第5回：検索してから回答する

キーワード検索でRAGの流れを学ぶ

今日のゴールと流れ

検索と生成を別の処理として説明する。

時間	内容
10分	資料全体を渡す方法の限界
20分	RAG、断片、出典の設計
20分	キーワード検索のコード
30分	検索の実装と失敗例の観察
10分	振り返り・提出

RAGとは

Retrieval-Augmented Generation：検索拡張生成。

質問に関係する情報を検索し、その情報を入力に加えて生成します。
この教材では、登録した資料の中を検索します。

RAGはインターネット検索だけを指す言葉ではありません。
ベクトル検索を使わなくても、検索して生成する構成を作れます。

準備と質問時の処理

準備するとき	質問を受けたとき
資料を集める	質問と検索語を受け取る
見出しごとに分割する	関連する断片を探す
出典と本文を保存する	選んだ断片をLLMへ渡す
内容を点検する	回答と参照資料を表示する

準備は資料更新時、検索と生成は質問ごとに行います。

1問で使う関数

質問：「SQLiteにデータを保存するには？」
検索語：「SQLite 保存」、上位件数：3

順番	関数	結果
1	search_keyword()	点数付きの資料断片
2	build_context()	参照用の文字列
3	ask_with_context()	資料を使った回答
4	画面の表示処理	回答と実際に取得した本文

検索語と、回答してほしい質問は別々に渡します。

チャンク：資料の断片

教材では `##` の見出しごとに分けます。

- 小さすぎると、主語や条件が失われる
- 大きすぎると、無関係な内容も一緒に選ばれる
- 見出し、ファイル名、本文を一緒に保存する
- 表や手順は、意味が途切れない区切りを考える

1000文字を超える見出しは、教材の読み込み処理がエラーにします。

見出しで分割する例

説明用の原本：

```
# 図書室の案内
## 開室時間
平日は16時まで利用できます。
## 貸出冊数
1人3冊まで借りられます。
```

本文のある `##` ごとに2つの断片になります。
どちらにも資料名「図書室の案内」と原本のファイル名を付けます。

分割で条件が離れる例

説明用の文章：「一般利用は16時まで。ただし試験期間は18時まで。」

分割	「試験期間は何時まで？」への影響
2文を同じ断片に保存	通常と例外を一緒に参照できる
例外を別の断片へ分離	通常の時間だけ選ばれる可能性がある

検索で選ばれた1断片だけを読んでも、条件が分かるかを確認します。
長さだけでなく、意味がつながる単位を考えます。

検索語と質問

入力欄	例
質問	SQLiteにデータを保存するには？
検索語	SQLite 保存

初学者向けに、日本語の単語分割は人が行います。
空白で区切った各単語が含まれるかを調べます。
自然文のまま検索すると、一致しない場合があります。

検索語を分けた結果

```
search_text = 'SQLite 保存'
words = search_text.lower().split()
# ['sqlite', '保存']
```

`lower()` で英字を小文字にし、`split()` で空白ごとに分けます。
「SQLiteに保存するには？」は、この方法では1つの長い検索語です。

質問欄は自然文、検索語欄は空白区切りの語として使い分けます。

検索のスコア

検索語が「SQLite 保存」の場合：

断片の本文	一致する単語数
SQLiteにデータを保存します	2
SQLiteを使います	1
CSSで色を指定します	0

同じ単語が何回出ても、この実装では1点です。
0点は除き、点数の高い順に並べます。

手計算する3つの断片

検索語は `sqlite` と `保存` です。次の全文を検索対象とします。

断片	説明用の検索対象テキスト
A	sqliteにデータを保存します
B	sqliteを使います
C	cssで色を指定します

A・B・Cの点数を予想してください。
実装では、資料名と見出しも検索対象テキストに含めます。

手計算の解答

断片	sqliteを含む	保存を含む	合計
A	1	1	2
B	1	0	1
C	0	0	0

Cは0点なので候補から除きます。
この点数は、質問の答えが本文にある確率ではありません。

forとifで数える

```
score = 0
for word in words:
    if word in text:
        score += 1
```

`words` は空白で分けた検索語のリスト。
`text` は資料名・見出し・本文をつないだ文字列です。
`lower()` で英字の大文字小文字の差を小さくします。

ループ中のscore

Aの本文 `sqliteにデータを保存します` を処理する例です。

タイミング	word	判定	score
開始	まだ取り出していない	未判定	0
1回目	sqlite	含まれる	1
2回目	保存	含まれる	2

次の断片Bを処理するときには、`score = 0` から始めます。
前の断片の点数を引き継ぐと、順位が変わってしまいます。

上位件数top_k

```
results.sort(key=get_score, reverse=True)
return results[:top_k]
```

`get_score()` は辞書からスコアを取り出す関数です。
`reverse=True` で大きい順に並べます。
`[:top_k]` は先頭から指定件数まで取り出す書き方です。

並べ替えと上位選択の結果

さきほどの説明用の断片では、候補はAが2点、Bが1点です。

top_k	返す断片	理由
1	A	先頭の1件
3	A、B	正の点数の候補は2件だけ
5	A、B	0点のCは候補に戻さない

top_kは最大件数です。指定した数だけ必ず返るわけではありません。

同点の資料の扱い

説明用にAとBがどちらも2点、top_kが1の場合を考えます。

このコードは点数の高い順に並べ、同点では元の順番を保ちます。
元の一覧はDBのID順なので、同点ならIDが小さい断片が先です。

先に並んだことだけでは、答えに必要な資料とは判断できません。
評価では、本文に必要な根拠が含まれるかを見ます。

検索結果をLLMへ渡す

```
chunks = search_keyword(request.search_text, request.top_k)
result = ask_with_context(question, chunks)
```

第4回の `all_chunks()` が、検索関数へ変わりました。
プロンプトを作る関数と、画面の出典表示はそのまま使います。

変更箇所が少ない理由を説明してみましょう。

検索後に渡す本文

説明用のAが資料ID 1として選ばれた場合：

```
[資料1] 説明用ノート / 保存
SQLiteにデータを保存します。
```

LLMには、この参照文と利用者の質問を渡します。
この教材の `build_context()` は検索スコアを参照文に含めません。
スコアは画面で、選択の理由を調べるために確認できます。

実習1：検索スコアの穴埋め

```
cd session05/exercise
python init_db.py
python main.py
```

`knowledge.py` の `TODO` を完成させます。
コメントを外し、仮の `raise` を削除します。
検索語「SQLite 保存」、上位件数3で質問します。

実習2：上位件数の比較

同じ質問と検索語で、上位件数だけを1、3、5に変えます。

- 必要な資料は選ばれたか
- 無関係な資料が増えたか
- 根拠が分散した質問に答えられたか
- APIモードで入力トークンがどう変わったか

一度に変える条件は1つにします。

検索が失敗した場合

「データが消えない仕組み」を探したいのに、
資料には「永続化」としか書かれていない場合があります。

キーワード検索は、表記の違いや言い換えが苦手なことがあります。
「SQLite」「保存」などの語を足すとどう変わるでしょうか。

この失敗を、次回の意味検索と比較します。

検索と生成の失敗を分ける

検索結果	回答	最初に調べる場所
必要な根拠がない	誤答	検索語・分割・件数
必要な根拠がある	誤答	指示・文脈・モデルの出力
根拠がない	回答を控える	検索の改善が必要か
根拠がある	正答	別の質問でも確かめる

検索結果を見ずに、プロンプトだけを変え続けないようにします。

失敗原因を考える練習

質問：「SQLiteへの保存後、何をすればよい？」

ケース	観察
A	commitを説明する資料がDBにない
B	資料はDBにあるが、検索結果に含まれない
C	資料を渡したが、回答がcommitを説明しない

最初に直す場所はどこでしょうか。

実習3：見つからない質問

検索語を、どの資料にもない `zzzz-no-match` にします。

教材では検索結果が0件なら、LLMを呼ばずに
「参照できる資料が見つかりません」と返します。

0件でない場合にも、答えが含まれるとは限りません。
その場合の回答保留は、モデルの出力を評価します。

0件と根拠不足の違い

検索結果	教材の処理	確認すること
0件	LLMを呼ばず固定の案内を返す	利用量が0か
1件以上だが答えなし	資料を渡して生成する	適切に回答を保留するか
必要な根拠あり	資料を渡して生成する	根拠に沿って回答するか

下の2行の生成品質は、APIモードで確認します。
デモの抜粋表示は、回答保留の評価には使いません。

資料の鮮度と出典

- 原本を編集したらDBを更新する
- 古い案内と新しい案内を混在させない
- 複数の資料が矛盾するときの方針を決める
- 日付が重要な題材では、更新日などのカラムを追加する

教材のIDは再登録で変わる可能性があります。
評価結果には原本ファイル名・見出しも記録します。

確認問題

1. RAGでは、モデル自体を学習し直している？
2. 上位5件にすれば、必ず上位1件より正確になる？
3. 検索結果0件なら、必ずLLMを呼ぶ必要がある？
4. キーワード検索が見逃しやすい質問は？

提出物と次回

提出：`knowledge.py` と `worksheet.md` のGitHub URL。

- 検索語と上位件数
- 選ばれた資料のファイル名・見出し・スコア
- 見つかった例、見つからなかった例
- 検索と生成のどちらを直すべきか

次回は、文章をベクトルにして検索します。